

# Vertex AI Vision 排隊偵測應用程式

來源：<https://codelabs.developers.google.com/vertex-ai-vision-queue-detection?hl=zh-tw>

步驟數：11

本程式碼研究室著重於建立端對端 Vertex AI Vision 應用程式，以監控零售儲存庫中的佇列偵測情境。我們會使用預先訓練的專業模型人數分析。您也會學到如何建立要在應用程式中擷取的視訊串流、如何建構及部署應用程式、如何透過 BigQuery 分析模型的 JSON 輸出內容，並在 Looker Studio 中以圖表呈現結果。

---

# 目錄

1. [1. 目標](#)
2. [2. 事前準備](#)
3. [3. 設定 Jupyter 筆記本](#)
4. [4. 設定筆記本來串流影片](#)
5. [5. 擷取影片檔案以進行串流](#)
6. [6. 建立應用程式](#)
7. [7. 將輸出內容連結至 BigQuery 資料表](#)
8. [8. 部署應用程式以供使用](#)
9. [9. 在儲存倉儲中搜尋影片內容](#)
10. [10. 使用 BigQuery 資料表為輸出內容加上註解及進行分析](#)
11. [11. 恭喜](#)

步驟 1 · 約 2 分鐘

# 1. 目標

## 總覽

本程式碼實驗室將著重於建立端對端 Vertex AI Vision 應用程式，使用零售影片片段監控排隊人數。我們將使用預先訓練的「Occupancy analytics」專用模型內建功能，擷取下列項目：

- 計算排隊人數。
- 計算櫃檯服務人數。

## 課程內容

- 如何在 Vertex AI Vision 中建立及部署應用程式
- 如何使用影片檔案設定 RTSP 串流，並從 Jupyter 筆記本使用 `vaictl` 將串流擷取至 Vertex AI Vision。
- 如何使用入住率分析模型及其不同功能。
- 瞭解如何搜尋儲存在 Vertex AI Vision 媒體倉儲中的影片。
- 如何將輸出內容連結至 BigQuery、編寫 SQL 查詢，從模型的 JSON 輸出內容中擷取洞察資料，並使用輸出內容標記原始影片和加上註解。

## 費用：

在 Google Cloud 上執行這個實驗室的總費用約為 \$2 美元。

---

## 2. 事前準備

### 建立專案並啟用 API：

1. 在 Google Cloud 控制台的專案選擇器頁面中，選取或[建立 Google Cloud 專案](#)。**注意：**如果您不打算保留在這項程序中建立的資源，請建立新專案，而不要選取現有專案。完成這些步驟後，您就可以刪除專案，並移除與該專案相關聯的所有資源。[前往專案選擇器](#)
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 啟用 Compute Engine、Vertex API、Notebook API 和 Vision AI API。[啟用 API](#)

### 建立服務帳戶：

1. 前往 Google Cloud 控制台中的「Create service account」(建立服務帳戶) 頁面。[前往「建立服務帳戶」](#)
2. 選取專案。
3. 在「Service account name」(服務帳戶名稱) 欄位中輸入名稱。Google Cloud 控制台會根據這個名稱填入「Service account ID」(服務帳戶 ID) 欄位。在「Service account description」(服務帳戶說明) 欄位中輸入說明。例如：快速入門導覽課程的服務帳戶。
4. 按一下 [建立並繼續]。
5. 如要提供專案存取權，請將下列角色授予服務帳戶：
  - **Vision AI > Vision AI 編輯器**
  - **「Compute Engine」 > 「Compute 執行個體管理員 (Beta 版)」**
  - **「BigQuery」 > 「BigQuery 管理員」。**

在「Select a role」(選取角色) 清單中，選取角色。如要新增其他角色，請按一下「新增其他角色」，然後新增其他角色。

**注意：**「Role」(角色) 欄位會影響服務帳戶在專案中可存取的資源。日後可以撤銷這些角色或授予其他角色。在正式環境中，請勿授予「擁有者」、「編輯者」或「檢視者」角色。而是根據需求，授予預先定義的角色或自訂角色。

6. 按一下「繼續」。
  7. 按一下「Done」(完成)，即完成建立服務帳戶。請勿關閉瀏覽器視窗，後續步驟會用到。
-

步驟 3 · 約 20 分鐘

### 3. 設定 Jupyter 筆記本

在 Occupancy Analytics 中建立應用程式之前，您必須先註冊串流，供應用程式日後使用。

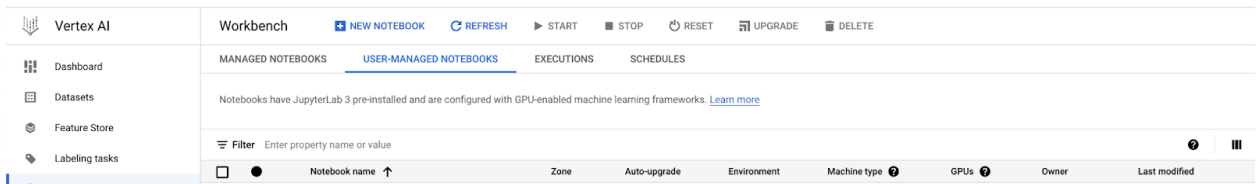
在本教學課程中，您會建立代管影片的 Jupyter Notebook 執行個體，並從該筆記本傳送串流影片資料。我們使用 Jupyter Notebook，因為這個工具可讓我們彈性執行殼層指令，以及在單一位置執行自訂的前/後處理程式碼，非常適合快速實驗。我們會使用這個筆記本執行下列操作：

1. 以背景程序執行 **rtsp** 伺服器
2. 以背景程序執行 **vaictl** 指令
3. 執行查詢和處理程式碼，分析入座率數據分析輸出內容

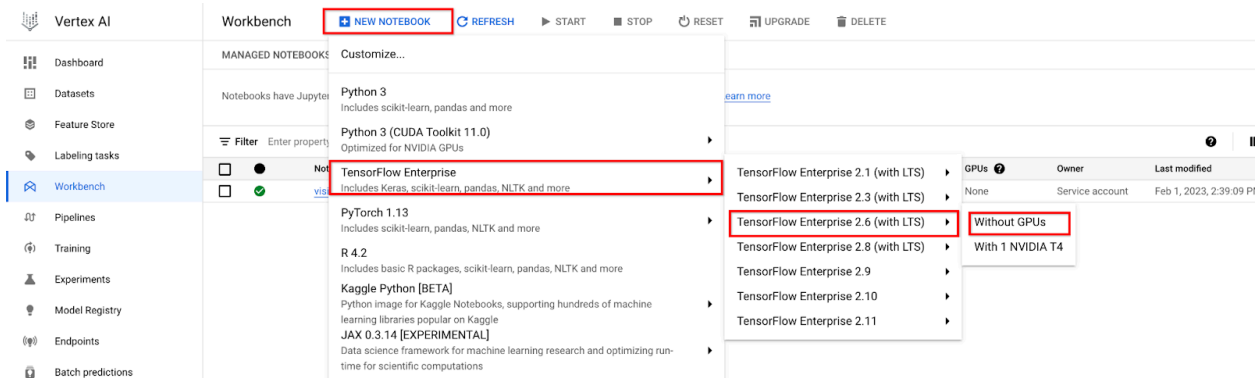
### 建立 Jupyter Notebook

如要從 Jupyter Notebook 執行個體傳送影片，第一步是使用上一步建立的服務帳戶建立筆記本。

1. 前往控制台的 Vertex AI 頁面。[前往 Vertex AI Workbench](#)
2. 按一下「使用者自行管理的筆記本」



3. 依序點按「New Notebook」>「Tensorflow Enterprise 2.6 (with LTS)」>「Without GPUs」



4. 輸入 Jupyter 筆記本的名稱。詳情請參閱「[資源命名慣例](#)」。

## New notebook

Notebook name \*

queue-detection-notebook

63-char limit with lowercase letters, digits, or '-' only. Must start with a letter. Cannot end with a '-'. [Learn more](#)

Region \*

us-west1 (Oregon)

Zone \*

us-west1-b

Some regions are restricted due to a policy set by your organization. [Learn more](#)

### Notebook properties

|                                                                                                  |                                                                                                            |
|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Environment     | TensorFlow Enterprise 2.6 (with LTS and Intel® MKL-DNN/MKL)                                                |
| Machine type                                                                                     | 4 vCPUs, 15 GB RAM                                                                                         |
| Boot disk                                                                                        | 100 GB Standard persistent disk                                                                            |
| Data disk                                                                                        | 100 GB Standard persistent disk                                                                            |
| Subnetwork                                                                                       | default(10.138.0.0/20)  |
| Permission                                                                                       | Compute Engine default service account                                                                     |
| Estimated cost  | \$112.91 monthly, \$0.155 hourly                                                                           |

ADVANCED OPTIONS

CANCEL

CREATE

- 點選「進階選項」
- 向下捲動至「權限」部分
- 取消勾選「使用 Compute Engine 預設服務帳戶」選項
- 新增上一個步驟中建立的服務帳戶電子郵件地址。然後按一下「建立」。

## Permission

JupyterLab access modes determine who can use a notebook instance and which credentials are used to call Google APIs. **This cannot be changed once the notebook is created.**

☒ Service account

Service account mode allows anyone who is granted the `iam.serviceAccounts.actAs` permission on the specified service account to access the JupyterLab UI. [Learn more](#)

☐ Single user only

Single user only mode restricts access to the user specified below. [Learn more](#)

☐ Use Compute Engine default service account 

Service account email \*

 Make sure this service account has sufficient API permissions. [Learn more](#)

☐ Enable Realtime Collaboration 

## Security

## Environment upgrade and system health



CANCEL

9. 執行個體建立完成後，按一下「OPEN JUPYTERLAB」。

步驟 4 · 約 15 分鐘

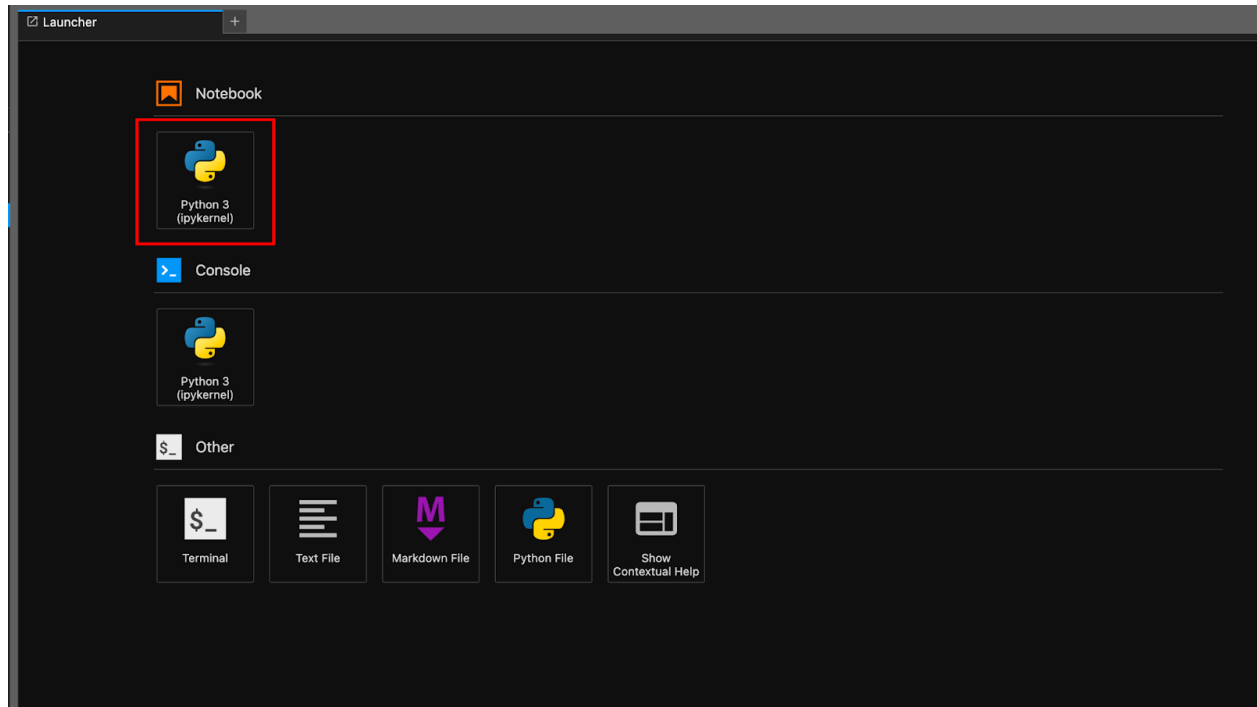
## 4. 設定筆記本來串流影片

在 Occupancy Analytics 中建立應用程式之前，您必須先註冊串流，供應用程式日後使用。

在本教學課程中，我們會使用 Jupyter Notebook 執行個體託管影片，並從 Notebook 終端機傳送串流影片資料。

### 下載 vaictl 指令列工具

1. 在開啟的 JupyterLab 執行個體中，從啟動器開啟「Notebook」。



2. 在筆記本儲存格中，使用下列指令下載 Vertex AI Vision (vaictl) 指令列工具、RTSP 伺服器指令列工具和 OpenCV 工具：

```
!wget -q https://github.com/aler9/rtsp-simple-server/releases/download/v0.20.4/rtsp-simple-server_v0.20.4_linux_amd64.tar.gz
!wget -q https://github.com/google/visionai/releases/download/v0.0.4/visionai_0.0-4_amd64.deb
!tar -xf rtsp-simple-server_v0.20.4_linux_amd64.tar.gz
!pip install opencv-python --quiet
!sudo apt-get -qq remove -y visionai
!sudo apt-get -qq install -y ./visionai_0.0-4_amd64.deb
!sudo apt-get -qq install -y ffmpeg
```

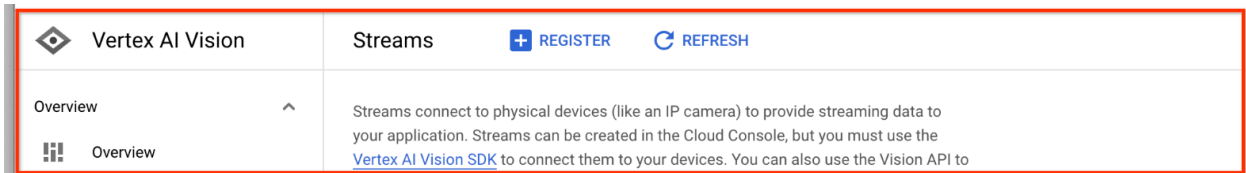


## 5. 擷取影片檔案以進行串流

使用必要的指令列工具設定筆記本環境後，即可複製影片樣本檔案，然後使用 `vaictl` 將影片資料串流至入住率數據分析應用程式。

### 註冊新的串流

1. 按一下 Vertex AI Vision 左側面板中的「串流」分頁標籤。
2. 按一下頂端的「註冊」按鈕



3. 在「Stream name」(串流名稱) 中輸入「queue-stream」
4. 在區域中，選擇與上一步建立 Notebooks 時選取的區域相同。
5. 按一下「Register」(註冊)

請稍待片刻，系統最多可能需要 1 小時才能完成註冊。

### 將影片樣本複製到 VM

1. 在筆記本中，使用下列 `wget` 指令複製影片樣本。

```
!wget -q https://github.com/vagrantism/interesting-  
datasets/raw/main/video/collective_activity/seq25_h264.mp4
```

### 從 VM 串流影片，並將資料擷取到串流中

1. 如要將這個本機影片檔案傳送至應用程式輸入串流，請在筆記本儲存格中使用下列指令。您必須進行下列變數替換：

- `PROJECT_ID`：您的 Google Cloud 專案 ID。
- `LOCATION`：您的地區 ID。例如 `us-central1`。詳情請參閱「[Cloud 據點](#)」一文。
- `LOCAL_FILE`：本機影片檔案的檔案名稱。例如 `seq25_h264.mp4`。

```
PROJECT_ID='<Your Google Cloud project ID>'  
LOCATION='<Your stream location>'  
LOCAL_FILE='seq25_h264.mp4'  
STREAM_NAME='queue-stream'
```

2. 啟動 `rtsp-simple-server`，透過 `rtsp` 通訊協定串流播放影片檔案

```
import os
import time
import subprocess

subprocess.Popen(["nohup", "./rtsp-simple-server"], stdout=open('rtsp_out.log', 'a'),
stderr=open('rtsp_err.log', 'a'), preexec_fn=os.setpgrp)
time.sleep(5)
```

### 3. 使用 ffmpeg 指令列工具，在 RTSP 串流中循環播放影片

```
subprocess.Popen(["nohup", "ffmpeg", "-re", "-stream_loop", "-1", "-i", LOCAL_FILE, "-c",
"copy", "-f", "rtsp", f"rtsp://localhost:8554/{LOCAL_FILE.split('.')[0]}"],
stdout=open('ffmpeg_out.log', 'a'), stderr=open('ffmpeg_err.log', 'a'),
preexec_fn=os.setpgrp)
time.sleep(5)
```

### 4. 使用 vaictl 指令列工具，將影片從 rtsp 伺服器 URI 串流至上個步驟中建立的 Vertex AI Vision 串流「queue-stream」。

```
subprocess.Popen(["nohup", "vaictl", "-p", PROJECT_ID, "-l", LOCATION, "-c", "application-
cluster-0", "--service-endpoint", "visionai.googleapis.com", "send", "rtsp", "to", "streams",
"queue-stream", "--rtsp-uri", f"rtsp://localhost:8554/{LOCAL_FILE.split('.')[0]}"],
stdout=open('vaictl_out.log', 'a'), stderr=open('vaictl_err.log', 'a'),
preexec_fn=os.setpgrp)
```

開始 vaictl 擷取作業後，影片可能需要約 100 秒才會顯示在資訊主頁。

串流擷取功能上線後，選取佇列串流，即可在 Vertex AI Vision 資訊主頁的「串流」分頁中查看影片動態消息。

[前往「串流」分頁](#)



queue-stream



EDIT



DELETE

|              |                          |
|--------------|--------------------------|
| Name         | queue-stream             |
| Stream ID    | queue-seq25              |
| Region       | us-central1              |
| Labels       | —                        |
| Last updated | Feb 2, 2023, 12:09:15 PM |

### Live view



You are watching with a delay of several seconds, due to time elapsed in video transmission and processing.

影片來源：Frames 資料集 [source](#)、影片 [source](#)

## 6. 建立應用程式

第一步是建立處理資料的應用程式。應用程式可視為自動化管道，可連結下列項目：

- **資料擷取**：將影片動態饋給擷取到串流中。
- **資料分析**：擷取資料後，即可加入 AI(電腦視覺) 模型。
- **資料儲存**：影片動態饋給的兩個版本 (原始串流和 AI 模型處理的串流) 可以儲存在媒體倉儲中。

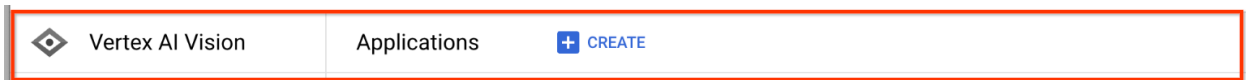
在 Google Cloud 控制台中，應用程式會以圖表表示。

### 建立空白的應用程式

您必須先建立空白應用程式，才能填入應用程式圖表。

在 Google Cloud 控制台中建立應用程式。

1. 前往 Google Cloud [控制台](#)。
2. 開啟 Vertex AI Vision 資訊主頁的「應用程式」分頁。[前往「應用程式」分頁](#)
3. 按一下「建立」按鈕。



4. 輸入「queue-app」做為應用程式名稱，然後選擇區域。
5. 點選「建立」。

### 新增應用程式元件節點

建立空白應用程式後，即可將三個節點新增至應用程式圖表：

1. **擷取節點**：串流資源，用於擷取從您在 Notebook 中建立的 RTSP 影片伺服器傳送的資料。
2. **處理節點**：根據擷取資料運作的入住率分析模型。
3. **儲存節點**：媒體倉儲，用於儲存處理過的影片，並做為中繼資料儲存空間。中繼資料儲存空間包括擷取的影片資料分析資訊，以及 AI 模型推斷的資訊。

在控制台將元件節點新增至應用程式。

1. 開啟 Vertex AI Vision 資訊主頁的「應用程式」分頁。[前往「應用程式」分頁](#)

系統會將您導向處理管道的圖表視覺化畫面。

### 新增資料擷取節點

1. 如要新增輸入串流節點，請在側邊選單的「Connectors」(連接器) 區段中選取「Streams」(串流) 選項。
2. 在隨即開啟的「串流」選單中，選取「來源」部分的「新增串流」。
3. 在「Add streams」(新增串流) 選單中，選擇「queue-stream」(佇列串流)。
4. 如要將串流新增至應用程式圖表，請按一下「新增串流」。

### 新增資料處理節點

1. 如要新增入住人數計數模型節點，請在側邊選單的「Specialized models」(專業模型) 區段中，選取「occupancy analytics」(入住人數分析) 選項。

2. 保留預設選取的「使用者」。如果已選取「車輛」，請取消勾選。

### Occupancy analytics



Uses advanced inputs like zones or lines to detect and count people and vehicles. Outputs bounding box coordinates, object labels and counts and confidence scores. [Learn more](#)

☒ People

☐ Vehicles

3. 在「進階選項」部分，按一下「建立活動區域/線條」

## Advanced options

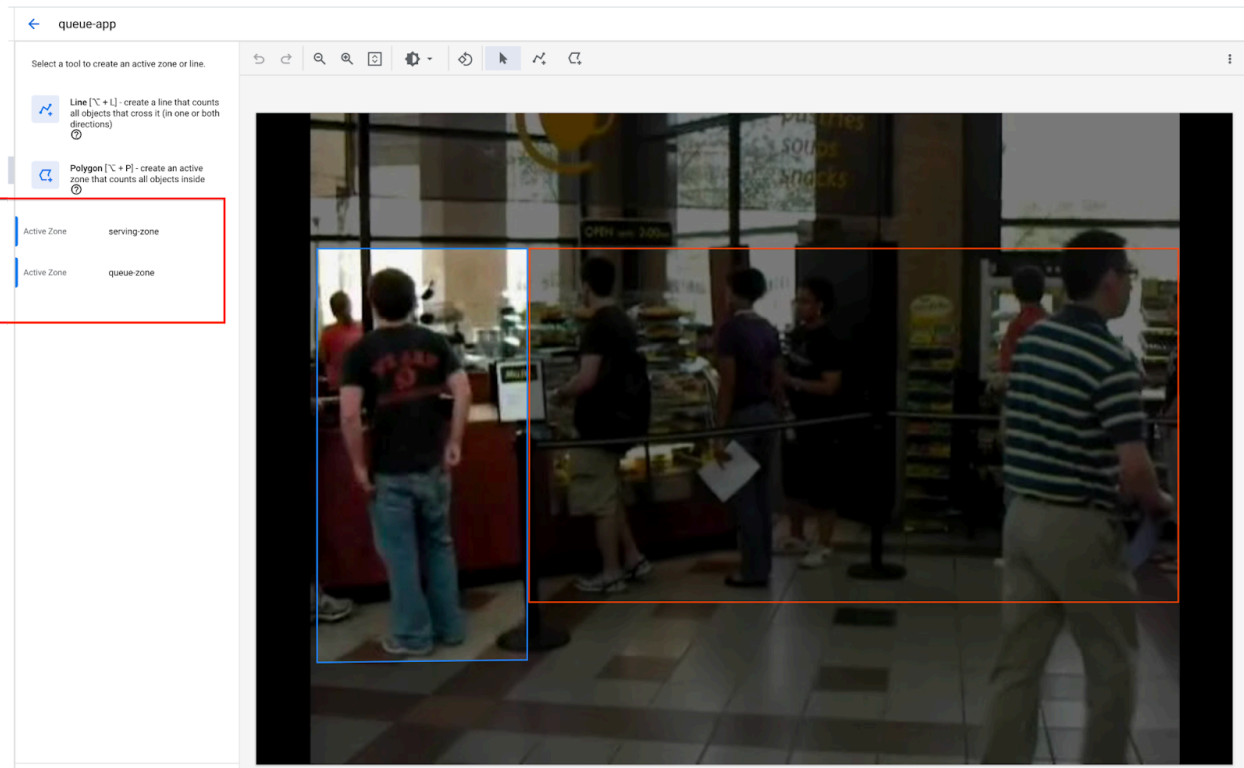


### No active zones or lines set up

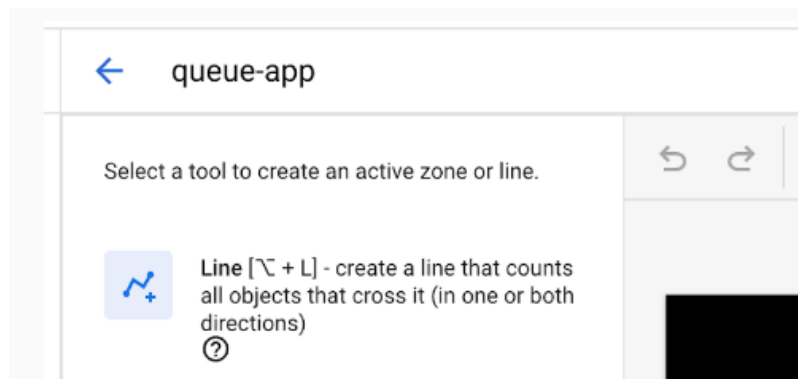
Draw polygons to specify where in your video streams you want to count vehicles and people. Or, draw lines to count line-crossing events.

[CREATE ACTIVE ZONES/ LINES](#)

4. 使用多邊形工具繪製活動區，計算該區域的人數。相應標示區域



5. 按一下頂端的返回箭頭。



6. 按一下核取方塊，新增停留時間設定，偵測擁擠情況。

### Dwell time

☒ Enable dwell time monitoring

Identify vehicles or people that have stayed within the frame (or active zone) for long periods of time. This can help with analyzing wait times, congestion, etc.



Specify at least one active zone for monitoring dwell time. The minimal dwell time is 10 seconds.

## 新增資料儲存節點

1. 如要新增輸出目的地 (儲存空間) 節點，請在側邊選單的「Connectors」(連接器) 區段中選取「Vision AI Warehouse」(Vision AI 倉儲) 選項。
2. 按一下「Vertex AI Warehouse」連接器開啟選單，然後按一下「Connect warehouse」。

3. 在「Connect Warehouse」(連結倉儲) 選單中，選取「Create new warehouse」(新建倉儲)。將倉儲命名為 **queue-warehouse**，並將 TTL 持續時間保留 14 天。
  4. 按一下「Create」(建立) 按鈕來新增倉儲。
-

## 7. 將輸出內容連結至 BigQuery 資料表

將 BigQuery 連接器新增至 Vertex AI Vision 應用程式後，所有已連結應用程式的模型輸出內容都會擷取至目標資料表。

您可以自行建立 BigQuery 資料表，並在應用程式中新增 BigQuery 連接器時指定該資料表，也可以讓 Vertex AI Vision 應用程式平台自動建立資料表。

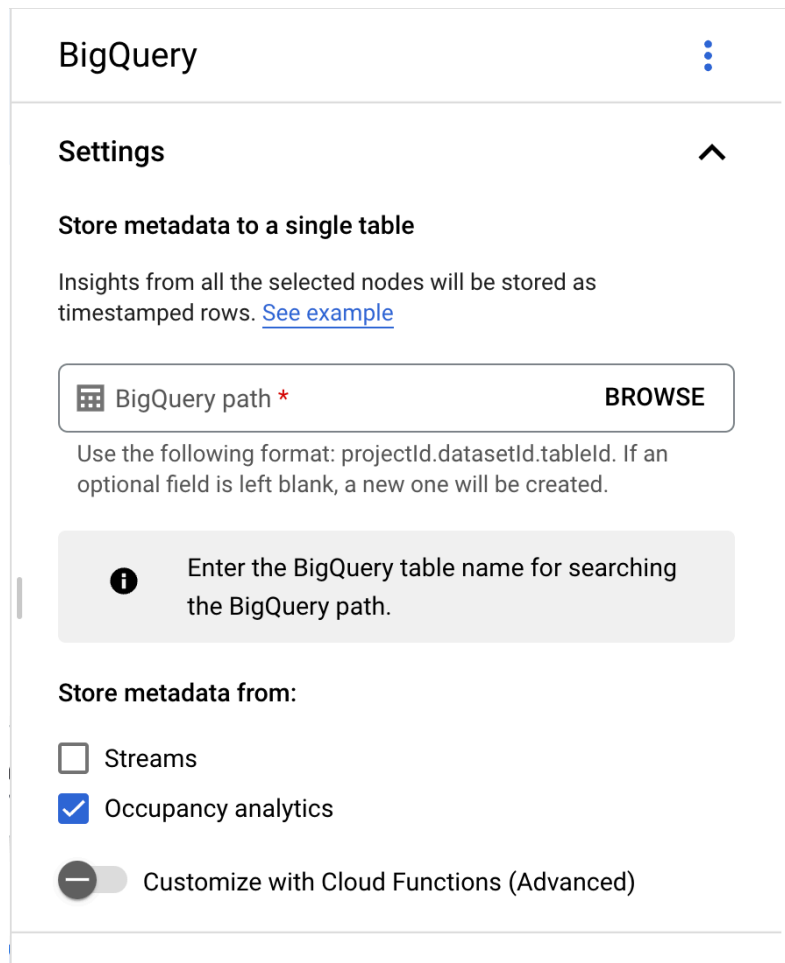
### 自動建立表格

如果讓 Vertex AI Vision 應用程式平台自動建立資料表，您可以在新增 BigQuery 連接器節點時指定這個選項。

如要使用自動建立資料表功能，必須符合下列資料集和資料表條件：

- 資料集：系統會自動建立名為「visionai\_dataset」的資料集。
- 資料表：系統會自動建立資料表名稱，格式為 visionai\_dataset.APPLICATION\_ID。
- 錯誤處理：
- 如果相同資料集下已有名稱相同的資料表，系統就不會自動建立資料表。

1. 開啟 Vertex AI Vision 資訊主頁的「應用程式」分頁。[前往「應用程式」分頁](#)
2. 從清單中選取應用程式名稱旁的「查看應用程式」。
3. 在應用程式建構工具頁面中，從「連接器」部分選取「BigQuery」。
4. 將「BigQuery 路徑」欄位留空。

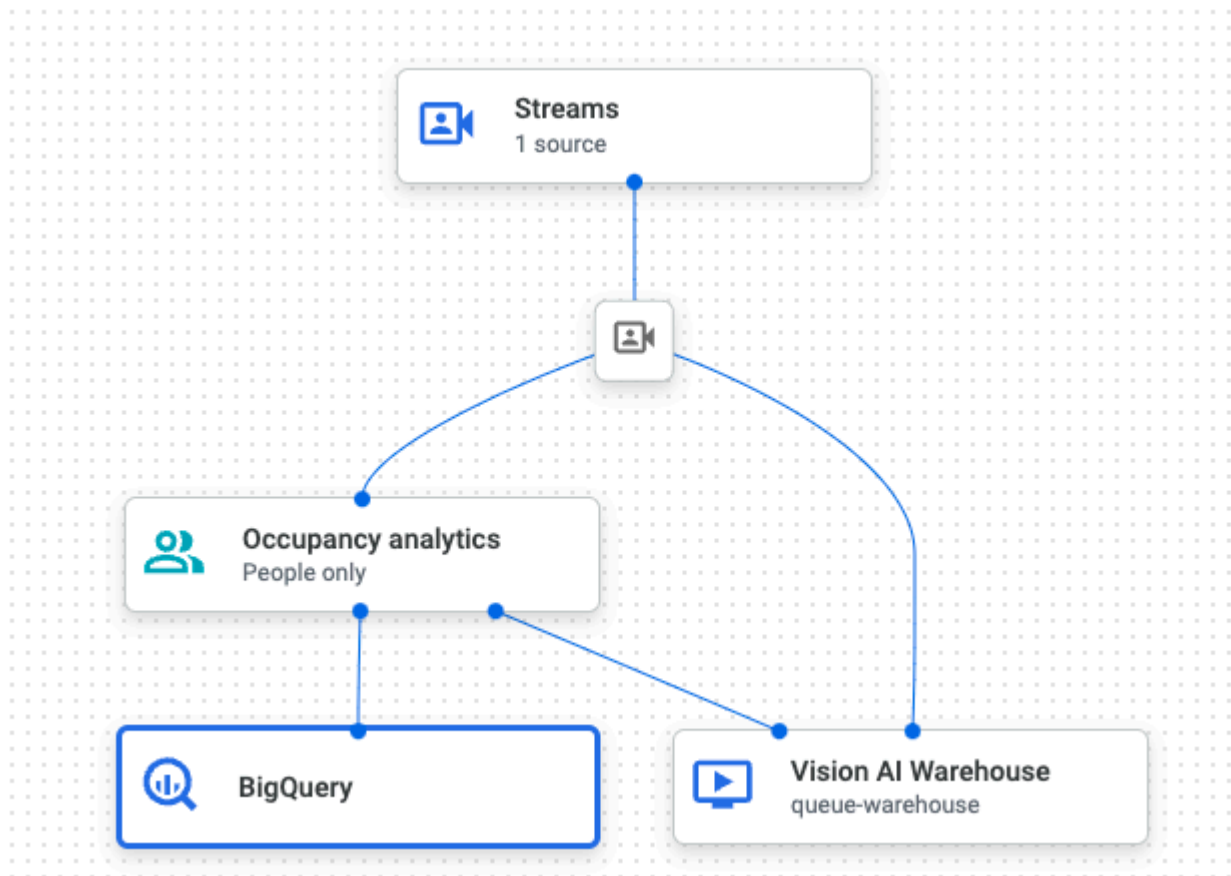


The screenshot shows the 'BigQuery' configuration panel. At the top, there's a title 'BigQuery' with a menu icon. Below it is a 'Settings' section with an expand/collapse arrow. The main heading is 'Store metadata to a single table', followed by a description: 'Insights from all the selected nodes will be stored as timestamped rows. [See example](#)'. There is a text input field for 'BigQuery path' with a red asterisk and a 'BROWSE' button. Below the field is a note: 'Use the following format: projectId.datasetId.tableId. If an optional field is left blank, a new one will be created.' A grey information box contains an 'i' icon and the text: 'Enter the BigQuery table name for searching the BigQuery path.' At the bottom, under 'Store metadata from:', there are two checkboxes: 'Streams' (unchecked) and 'Occupancy analytics' (checked). A toggle switch for 'Customize with Cloud Functions (Advanced)' is shown at the very bottom, currently turned off.

5. 在「儲存來源的中繼資料」中，只選取「車輛乘載人數分析」，並取消勾選串流。

最終的應用程式圖表應如下所示：

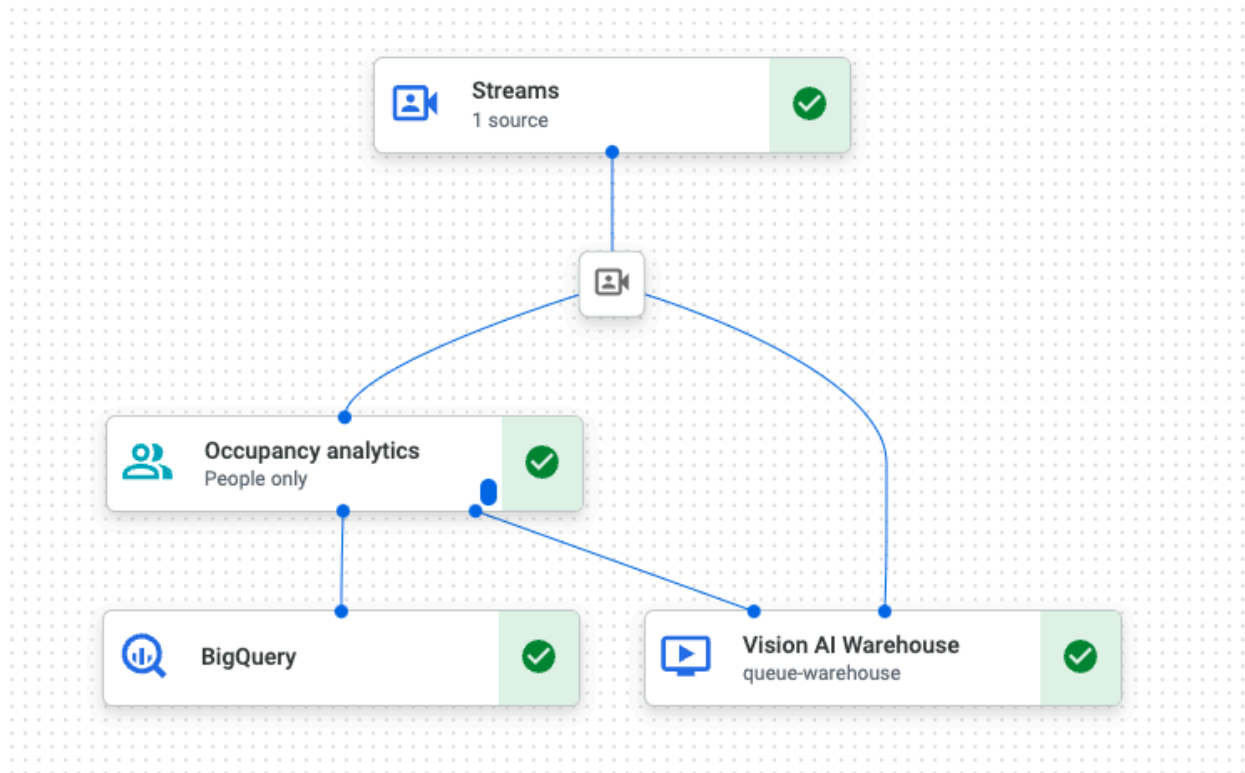




## 8. 部署應用程式以供使用

使用所有必要元件建構端對端應用程式後，最後一個步驟就是部署應用程式。

1. 開啟 Vertex AI Vision 資訊主頁的「應用程式」分頁。[前往「應用程式」分頁](#)
2. 在清單中，選取「queue-app」應用程式旁的「查看應用程式」。
3. 在「Studio」頁面中，按一下「Deploy」按鈕。
4. 在隨後的確認對話方塊中，按一下「Deploy」。部署作業可能需要幾分鐘才能完成。部署作業完成後，節點旁會顯示綠色勾號。



## 9. 在儲存倉儲中搜尋影片內容

將影片資料擷取至處理應用程式後，即可查看分析結果，並根據入住率數據分析資訊搜尋資料。

1. 開啟 Vertex AI Vision 資訊主頁的「倉庫」分頁。[前往「倉庫」分頁](#)
2. 在清單中找出 queue-warehouse 倉庫，然後點按「查看資產」。
3. 在「人數」部分，將「最小值」設為 1，並將「最大值」設為 5。
4. 如要篩選儲存在 Vertex AI Vision Media Warehouse 中的已處理影片資料，請按一下「搜尋」。

← queue-warehouse

VIEW DETAILS

Filters

Specify how you want to refine your search results. [Learn more](#)

Stream name

Enter different stream names as chips.

Date range

Vehicle Count

Min

—

Max

+ ADD RANGE (OR)

People Count

Min

1

—

Max

5

+ ADD RANGE (OR)

Search Criteria

+ ADD CRITERIA (AND)

Stored assets are deleted after 14 days. To change the time to live (TTL) length, click Edit.

EDIT

queue-stream

Google Cloud 控制台會顯示符合搜尋條件的已儲存影片資料。

## 10. 使用 BigQuery 資料表為輸出內容加上註解及進行分析

1. 在筆記本中，於儲存格中初始化下列變數。

```
DATASET_ID='vision_ai_dataset'  
bq_table=f'{PROJECT_ID}.{DATASET_ID}.queue-app'  
frame_buffer_size=10000  
frame_buffer_error_milliseconds=5  
dashboard_update_delay_seconds=3  
rtsp_url='rtsp://localhost:8554/seq25_h264'
```

2. 現在，我們將使用下列程式碼，從 RTSP 串流擷取影格：

```
import cv2  
import threading  
from collections import OrderedDict  
from datetime import datetime, timezone  
  
frame_buffer = OrderedDict()  
frame_buffer_lock = threading.Lock()  
  
stream = cv2.VideoCapture(rtsp_url)  
def read_frames(stream):  
    global frames  
    while True:  
        ret, frame = stream.read()  
        frame_ts = datetime.now(timezone.utc).timestamp() * 1000  
        if ret:  
            with frame_buffer_lock:  
                while len(frame_buffer) >= frame_buffer_size:  
                    _ = frame_buffer.popitem(last=False)  
                frame_buffer[frame_ts] = frame  
  
frame_buffer_thread = threading.Thread(target=read_frames, args=(stream,))  
frame_buffer_thread.start()  
print('Waiting for stream initialization')  
while not list(frame_buffer.keys()): pass  
print('Stream Initialized')
```

3. 從 BigQuery 資料表擷取資料時間戳記和註解資訊，並建立目錄來儲存擷取的影格圖片：

```

from google.cloud import bigquery
import pandas as pd

client = bigquery.Client(project=PROJECT_ID)

query = f"""
SELECT MAX(ingestion_time) AS ts
FROM `{bq_table}`
"""

bq_max_ingest_ts_df = client.query(query).to_dataframe()
bq_max_ingest_epoch = str(int(bq_max_ingest_ts_df['ts'][0].timestamp()*1000000))
bq_max_ingest_ts = bq_max_ingest_ts_df['ts'][0]
print('Preparing to pull records with ingestion time >', bq_max_ingest_ts)
if not os.path.exists(bq_max_ingest_epoch):
    os.makedirs(bq_max_ingest_epoch)
print('Saving output frames to', bq_max_ingest_epoch)

```

4. 使用下列程式碼為影格加上註解：

```

import json
import base64
import numpy as np
from IPython.display import Image, display, HTML, clear_output

im_width = stream.get(cv2.CAP_PROP_FRAME_WIDTH)
im_height = stream.get(cv2.CAP_PROP_FRAME_HEIGHT)

dashdelta = datetime.now()
framedata = {}
cntext = lambda x: {y['entity']['labelString']: y['count'] for y in x}
try:
    while True:
        try:
            annotations_df = client.query(f'''
                SELECT ingestion_time, annotation
                FROM `{bq_table}`
                WHERE ingestion_time > TIMESTAMP("{bq_max_ingest_ts}")
            ''').to_dataframe()
        except ValueError as e:
            continue
        bq_max_ingest_ts = annotations_df['ingestion_time'].max()
        for _, row in annotations_df.iterrows():
            with frame_buffer_lock:
                frame_ts = np.asarray(list(frame_buffer.keys()))
                delta_ts = np.abs(frame_ts - (row['ingestion_time'].timestamp() * 1000))
                delta_tx_idx = delta_ts.argmin()
                closest_ts_delta = delta_ts[delta_tx_idx]
                closest_ts = frame_ts[delta_tx_idx]
                if closest_ts_delta > frame_buffer_error_milliseconds: continue
                image = frame_buffer[closest_ts]
            annotations = json.loads(row['annotation'])
            for box in annotations['identifiedBoxes']:
                image = cv2.rectangle(
                    image,
                    (
                        int(box['normalizedBoundingBox']['xmin']*im_width),
                        int(box['normalizedBoundingBox']['ymin']*im_height)
                    ),
                    (
                        int((box['normalizedBoundingBox']['xmin'] + box['normalizedBoundingBox']
                            ['width'])*im_width),
                        int((box['normalizedBoundingBox']['ymin'] + box['normalizedBoundingBox']
                            ['height'])*im_height)
                    ),
                    (255, 0, 0), 2
                )
            img_filename = f"{bq_max_ingest_epoch}/{row['ingestion_time'].timestamp() * 1000}.png"
            cv2.imwrite(img_filename, image)
            binimg = base64.b64encode(cv2.imencode('.jpg', image)[1]).decode()
            curr_framedata = {
                'path': img_filename,

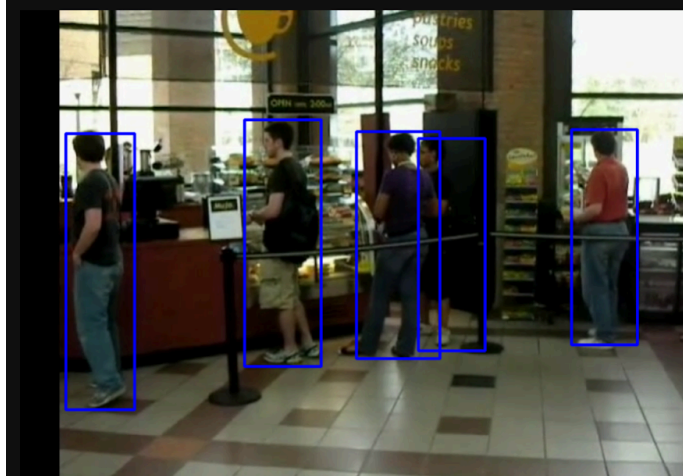
```

```

'timestamp_error': closest_ts_delta,
'counts': {
    **{
        k['annotation']['displayName'] : cntext(k['counts'])
        for k in annotations['stats']['activeZoneCounts']
    },
    'full-frame': cntext(annotations['stats']['fullFrameCount'])
}
}
framedata[img_filename] = curr_framedata
if (datetime.now() - dashdelta).total_seconds() > dashboard_update_delay_seconds:
    dashdelta = datetime.now()
    clear_output()
    display(HTML(f'''
        <h1>Queue Monitoring Application</h1>
        <p>Live Feed of the queue camera:</p>
        <p></a></p>
        <table border="1" cellpadding="1" cellspacing="1" style="width: 500px;">
            <caption>Current Model Outputs</caption>
            <thead>
                <tr><th scope="row">Metric</th><th scope="col">Value</th></tr>
            </thead>
            <tbody>
                <tr><th scope="row">Serving Area People Count</th><td>{curr_framedata['counts']
['serving-zone']['Person']}

```

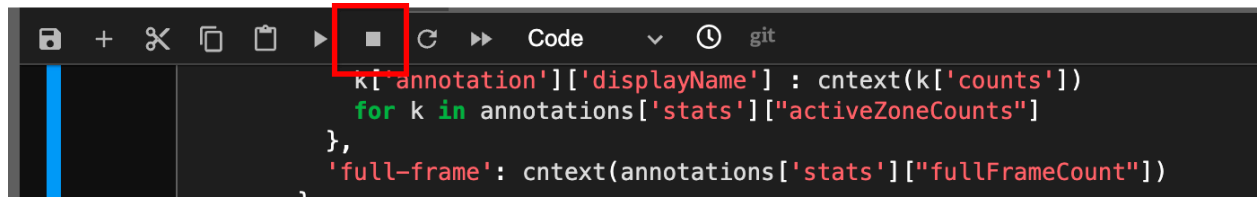
Live Feed of the queue camera:



Current Model Outputs

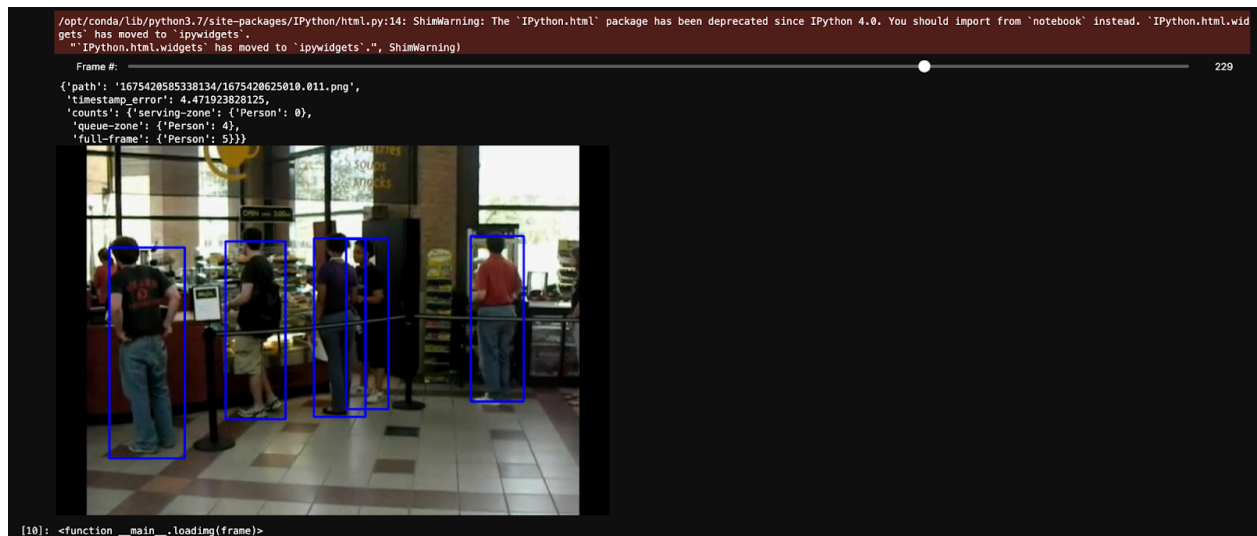
| Metric                     | Value          |
|----------------------------|----------------|
| Serving Area People Count  | 0              |
| Queueing Area People Count | 3              |
| Total Area People Count    | 5              |
| Timestamp Error            | 1.353759765625 |

5. 使用筆記本選單列中的「停止」按鈕停止註解工作



6. 您可以使用下列程式碼，重新造訪個別影格：

```
from IPython.html.widgets import Layout, interact, IntSlider
imgs = sorted(list(framedata.keys()))
def loading(frame):
    display(framedata[imgs[frame]])
    display(Image(open(framedata[imgs[frame]]['path'], 'rb').read()))
interact(loading, frame=IntSlider(
    description='Frame #:',
    value=0,
    min=0, max=len(imgs)-1, step=1,
    layout=Layout(width='100%')))
```





步驟 11 · 約 10 分鐘

## 11. 恭喜

恭喜，您已完成本實驗室！

### 清理

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

刪除個別資源

### 資源

<https://cloud.google.com/vision-ai/docs/overview>

<https://cloud.google.com/vision-ai/docs/occupancy-count-tutorial>

### 授權

### 問卷調查

---